

This is a particular case of a *similarity transform* of the matrix $\mathbf{A}$,

$$\mathbf{A} \rightarrow \mathbf{Z}^{-1} \cdot \mathbf{A} \cdot \mathbf{Z} \quad (11.0.12)$$

for some transformation matrix $\mathbf{Z}$. Similarity transformations play a crucial role in the computation of eigenvalues, because they leave the eigenvalues of a matrix unchanged. This is easily seen from

$$\begin{aligned} \det |\mathbf{Z}^{-1} \cdot \mathbf{A} \cdot \mathbf{Z} - \lambda \mathbf{1}| &= \det |\mathbf{Z}^{-1} \cdot (\mathbf{A} - \lambda \mathbf{1}) \cdot \mathbf{Z}| \\ &= \det |\mathbf{Z}| \det |\mathbf{A} - \lambda \mathbf{1}| \det |\mathbf{Z}^{-1}| \\ &= \det |\mathbf{A} - \lambda \mathbf{1}| \end{aligned} \quad (11.0.13)$$

Equation (11.0.11) shows that any matrix with complete eigenvectors (which includes all normal matrices and “most” random nonnormal ones) can be diagonalized by a similarity transformation, that the columns of the transformation matrix that effects the diagonalization are the right eigenvectors, and that the rows of its inverse are the left eigenvectors.

For real symmetric matrices, the eigenvectors are real and orthonormal, so the transformation matrix is orthogonal. The similarity transformation is then also an *orthogonal transformation* of the form

$$\mathbf{A} \rightarrow \mathbf{Z}^T \cdot \mathbf{A} \cdot \mathbf{Z} \quad (11.0.14)$$

While real nonsymmetric matrices can be diagonalized in their usual case of complete eigenvectors, the transformation matrix is not necessarily real. It turns out, however, that a real similarity transformation can “almost” do the job. It can reduce the matrix down to a form with little two-by-two blocks along the diagonal and all other elements zero. Each two-by-two block corresponds to a complex-conjugate pair of complex eigenvalues. We will see this idea exploited in some routines given later in the chapter.

The “grand strategy” of virtually all modern eigensystem routines is to nudge the matrix $\mathbf{A}$ toward diagonal form by a sequence of similarity transformations,

$$\begin{aligned} \mathbf{A} &\rightarrow \mathbf{P}_1^{-1} \cdot \mathbf{A} \cdot \mathbf{P}_1 \rightarrow \mathbf{P}_2^{-1} \cdot \mathbf{P}_1^{-1} \cdot \mathbf{A} \cdot \mathbf{P}_1 \cdot \mathbf{P}_2 \\ &\rightarrow \mathbf{P}_3^{-1} \cdot \mathbf{P}_2^{-1} \cdot \mathbf{P}_1^{-1} \cdot \mathbf{A} \cdot \mathbf{P}_1 \cdot \mathbf{P}_2 \cdot \mathbf{P}_3 \rightarrow \text{etc.} \end{aligned} \quad (11.0.15)$$

If we get all the way to diagonal form, then the eigenvectors are the columns of the accumulated transformation

$$\mathbf{X}_R = \mathbf{P}_1 \cdot \mathbf{P}_2 \cdot \mathbf{P}_3 \cdot \dots \quad (11.0.16)$$

Sometimes we do not want to go all the way to diagonal form. For example, if we are interested only in eigenvalues, not eigenvectors, it is enough to transform the matrix $\mathbf{A}$ to be triangular, with all elements below (or above) the diagonal zero. In this case the diagonal elements are already the eigenvalues, as you can see by mentally evaluating (11.0.2) using expansion by minors.

There are two rather different sets of techniques for implementing the grand strategy (11.0.15). It turns out that they work rather well in combination, so most modern eigensystem routines use both. The first set of techniques constructs individual $\mathbf{P}_i$ ’s as explicit “atomic” transformations designed to perform specific tasks,

for example zeroing a particular off-diagonal element (Jacobi transformation, §11.1), or a whole particular row or column (Householder transformation, §11.3; elimination method, §11.6). In general, a finite sequence of these simple transformations cannot completely diagonalize a matrix. There are then two choices: either use the finite sequence of transformations to go most of the way (e.g., to some special form like *tridiagonal* or *Hessenberg*; see §11.3 and §11.6 below) and follow up with the second set of techniques about to be mentioned; or else iterate the finite sequence of simple transformations over and over until the deviation of the matrix from diagonal is negligibly small. This latter approach is conceptually simplest, so we will discuss it in the next section; however, for N greater than ~ 10 , it is computationally inefficient by a roughly constant factor ~ 5 .

The second set of techniques, called *factorization methods*, is more subtle. Suppose that the matrix $\mathbf{A}$ can be factored into a left factor $\mathbf{F}_L$ and a right factor $\mathbf{F}_R$. Then

$$\mathbf{A} = \mathbf{F}_L \cdot \mathbf{F}_R \quad \text{or equivalently} \quad \mathbf{F}_L^{-1} \cdot \mathbf{A} = \mathbf{F}_R \quad (11.0.17)$$

If we now multiply back together the factors in the reverse order and use the second equation in (11.0.17), we get

$$\mathbf{F}_R \cdot \mathbf{F}_L = \mathbf{F}_L^{-1} \cdot \mathbf{A} \cdot \mathbf{F}_L \quad (11.0.18)$$

which we recognize as having effected a similarity transformation on $\mathbf{A}$ with the transformation matrix being $\mathbf{F}_L$! In §11.4 and §11.7 we will discuss the *QR method* that exploits this idea.

Factorization methods also do not converge exactly in a finite number of transformations. But the better ones do converge rapidly and reliably, and, when following an appropriate initial reduction by simple similarity transformations, they are generally the methods of choice.

11.0.4 “Eigenpackages of Canned Eigenroutines”

You have probably gathered by now that the solution of eigensystems is a fairly complicated business. It is. It is one of the few subjects covered in this book for which we do *not* recommend that you avoid canned routines. On the contrary, the purpose of this chapter is precisely to give you some appreciation of what is going on inside such canned routines, so that you can make intelligent choices about using them, and intelligent diagnoses when something goes wrong.

You will find that almost all canned routines in use nowadays trace their ancestry back to routines published in Wilkinson and Reinsch’s *Handbook for Automatic Computation, Vol. II, Linear Algebra* [2]. This excellent reference, containing papers by a number of authors, is the Bible of the field. A public-domain implementation of the *Handbook* routines in Fortran is the EISPACK set of programs [3]. The routines in this chapter are translations of either the *Handbook* or EISPACK routines, so understanding these will take you a lot of the way toward understanding those canonical packages.

The successor to EISPACK is LAPACK [4], which also includes the linear algebra routines of LINPACK. This is a Fortran package in which a lot of attention has been paid to efficient execution on modern machines. A C translation is available as CLAPACK.

IMSL [5] and NAG [6] each provide proprietary implementations, in Fortran and C, of what are essentially the *Handbook* routines.

A good “eigenpackage” will provide separate routines, or separate paths through sequences of routines, for the following desired calculations:

- all eigenvalues and no eigenvectors
- all eigenvalues and some corresponding eigenvectors
- all eigenvalues and all corresponding eigenvectors

The purpose of these distinctions is to save compute time and storage; it is wasteful to calculate eigenvectors that you don’t need. Often one is interested only in the eigenvectors corresponding to the largest few eigenvalues, or largest few in magnitude, or few that are negative. The method usually used to calculate “some” eigenvectors is typically more efficient than calculating all eigenvectors if you desire fewer than about a quarter of the eigenvectors.

A good eigenpackage also provides separate paths for each of the above calculations for each of the following special forms of the matrix:

- real, symmetric, tridiagonal
- real, symmetric, banded (only a small number of sub- and superdiagonals are nonzero)
- real, symmetric
- real, nonsymmetric
- complex, Hermitian
- complex, non-Hermitian

Again, the purpose of these distinctions is to save time and storage by using the *least* general routine that will serve in any particular application.

In this chapter, as a bare introduction, we give good routines for the following paths:

- all eigenvalues and eigenvectors of a real symmetric, tridiagonal matrix (§11.4)
- all eigenvalues and eigenvectors of a real symmetric, matrix (§11.1 – §11.4)
- all eigenvalues and eigenvectors of a complex Hermitian matrix (§11.5)
- all eigenvalues and eigenvectors of a real nonsymmetric matrix (§11.6 – §11.7)

We also discuss, in §11.8, how to obtain some eigenvectors of general matrices by the method of inverse iteration.

11.0.5 Generalized and Nonlinear Eigenvalue Problems

Many eigenpackages also deal with the so-called *generalized eigenproblem* [7],

$$\mathbf{A} \cdot \mathbf{x} = \lambda \mathbf{B} \cdot \mathbf{x} \quad (11.0.19)$$

where $\mathbf{A}$ and $\mathbf{B}$ are both matrices. Most such problems, where $\mathbf{B}$ is nonsingular, can be handled by the equivalent

$$(\mathbf{B}^{-1} \cdot \mathbf{A}) \cdot \mathbf{x} = \lambda \mathbf{x} \quad (11.0.20)$$

Often $\mathbf{A}$ and $\mathbf{B}$ are symmetric and $\mathbf{B}$ is positive-definite. The matrix $\mathbf{B}^{-1} \cdot \mathbf{A}$ in (11.0.20) is not symmetric, but we can recover a symmetric eigenvalue problem by using the Cholesky decomposition $\mathbf{B} = \mathbf{L} \cdot \mathbf{L}^T$ of §2.9. Multiplying equation (11.0.19) by $\mathbf{L}^{-1}$, we get

$$\mathbf{C} \cdot (\mathbf{L}^T \cdot \mathbf{x}) = \lambda (\mathbf{L}^T \cdot \mathbf{x}) \quad (11.0.21)$$

where

$$\mathbf{C} = \mathbf{L}^{-1} \cdot \mathbf{A} \cdot (\mathbf{L}^{-1})^T \quad (11.0.22)$$

The matrix $\mathbf{C}$ is symmetric and its eigenvalues are the same as those of the original problem (11.0.19); its eigenfunctions are $\mathbf{L}^T \cdot \mathbf{x}$. The efficient way to form $\mathbf{C}$ is first to solve the equation

$$\mathbf{Y} \cdot \mathbf{L}^T = \mathbf{A} \quad (11.0.23)$$

for the lower triangle of the matrix $\mathbf{Y}$. Then solve

$$\mathbf{L} \cdot \mathbf{C} = \mathbf{Y} \quad (11.0.24)$$

for the lower triangle of the symmetric matrix $\mathbf{C}$.

Another generalization of the standard eigenvalue problem is to problems non-linear in the eigenvalue λ , for example,

$$(\mathbf{A}\lambda^2 + \mathbf{B}\lambda + \mathbf{C}) \cdot \mathbf{x} = 0 \quad (11.0.25)$$

This can be turned into a linear problem by introducing an additional unknown eigenvector $\mathbf{y}$ and solving the $2N \times 2N$ eigensystem

$$\begin{pmatrix} 0 & \mathbf{1} \\ -\mathbf{A}^{-1} \cdot \mathbf{C} & -\mathbf{A}^{-1} \cdot \mathbf{B} \end{pmatrix} \cdot \begin{pmatrix} \mathbf{x} \\ \mathbf{y} \end{pmatrix} = \lambda \begin{pmatrix} \mathbf{x} \\ \mathbf{y} \end{pmatrix} \quad (11.0.26)$$

This technique generalizes to higher-order polynomials in λ . A polynomial of degree M produces a linear $MN \times MN$ eigensystem (see [8]).

11.0.6 Relation to Singular Value Decomposition

The factorization of a matrix $\mathbf{A}$ by the use of its eigenvectors and eigenvalues, equation (11.0.10), seems similar to singular value decomposition (SVD), as was discussed in §2.6. Is it the same thing? In general, no. A first obvious difference is that SVD is not restricted to square matrices, while eigendecomposition is. But what if $\mathbf{A}$ is square? Are the two decompositions then identical?

In general, still no. The difference has to do with what is orthogonal to what. If for a square matrix $\mathbf{A}$ we write the two decompositions (cf. equation 2.6.1 or 2.6.4 and equation 11.0.10),

$$\mathbf{A} = \mathbf{U} \cdot \text{diag}(w_0 \dots w_{N-1}) \cdot \mathbf{V}^T = \mathbf{X}_R \cdot \text{diag}(\lambda_0 \dots \lambda_{N-1}) \cdot \mathbf{X}_L \quad (11.0.27)$$

then for SVD the columns of $\mathbf{U}$ are mutually orthonormal, as are the columns of $\mathbf{V}$. There is no particular orthonormality *between* $\mathbf{U}$ and $\mathbf{V}$. For the eigendecomposition, the situation is the reverse: The rows of $\mathbf{X}_L$ are orthogonal to the columns of $\mathbf{X}_R$ (except for those corresponding to the same eigenvalue), but there is no particular orthogonality among the rows or columns of $\mathbf{X}_L$ or the rows or columns of $\mathbf{X}_R$. The two decompositions in equation (11.0.27) are just, in general, different!

However, the difference disappears when $\mathbf{A}$ is symmetric (or, if complex, Hermitian). In that case, equation (11.0.27) becomes

$$\mathbf{A} = \mathbf{V} \cdot \text{diag}(w_0 \dots w_{N-1}) \cdot \mathbf{V}^T = \mathbf{X}_R \cdot \text{diag}(\lambda_0 \dots \lambda_{N-1}) \cdot \mathbf{X}_R^T \quad (11.0.28)$$

and the fact that each decomposition is unique implies

$$\mathbf{V} = \mathbf{U} = \mathbf{X}_R = \mathbf{X}_L^T \quad (11.0.29)$$

and

$$\lambda_i = w_i, \quad i = 0, \dots, N-1 \quad (11.0.30)$$

That is, the (left and right) eigenvectors are the columns of any of the matrices listed in equation (11.0.29), and the corresponding eigenvalues and singular values are identical.

From a general matrix $\mathbf{A}$, not necessarily even square, one can form the two symmetric matrices $\mathbf{A}^T \cdot \mathbf{A}$ and $\mathbf{A} \cdot \mathbf{A}^T$. You can work out from equation (11.0.27) that the eigenvalues of either of these two matrices are squares of singular values of $\mathbf{A}$. However, this doesn't tell you about the eigenvalues of $\mathbf{A}$: The matrix whose eigenvalues are the squares of the eigenvalues of $\mathbf{A}$ is the unrelated matrix $\mathbf{A} \cdot \mathbf{A}$, not $\mathbf{A}^T \cdot \mathbf{A}$ or $\mathbf{A} \cdot \mathbf{A}^T$.

CITED REFERENCES AND FURTHER READING:

- Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), Chapter 6.[1]
- Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer).[2]
- Smith, B.T., et al. 1976, *Matrix Eigensystem Routines — EISPACK Guide*, 2nd ed., vol. 6 of *Lecture Notes in Computer Science* (New York: Springer).[3]
- Anderson, E., et al. 1999, *LAPACK User's Guide*, 3rd ed. (Philadelphia: S.I.A.M.). Online with software at 2007+, <http://www.netlib.org/lapack>.[4]
- IMSL Math/Library Users Manual* (Houston: IMSL Inc.), see 2007+, <http://www.vni.com/products/ims1>.[5]
- NAG Fortran Library* (Oxford, UK: Numerical Algorithms Group), see 2007+, <http://www.nag.co.uk>, Chapter F02.[6]
- Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), §7.7.[7]
- Wilkinson, J.H. 1965, *The Algebraic Eigenvalue Problem* (New York: Oxford University Press).[8]
- Acton, F.S. 1970, *Numerical Methods That Work*, 1990, corrected edition (Washington, DC: Mathematical Association of America), Chapter 13.
- Horn, R.A., and Johnson, C.R. 1985, *Matrix Analysis* (Cambridge: Cambridge University Press).

11.1 Jacobi Transformations of a Symmetric Matrix

The Jacobi method consists of a sequence of orthogonal similarity transformations of the form of equation (11.0.15). Each transformation (a *Jacobi rotation*) is just a plane rotation designed to annihilate one of the off-diagonal matrix elements. Successive transformations undo previously set zeros, but the off-diagonal elements nevertheless get smaller and smaller, until the matrix is diagonal to machine precision. Accumulating the product of the transformations as you go gives the matrix of

eigenvectors, equation (11.0.16), while the elements of the final diagonal matrix are the eigenvalues.

The Jacobi method is absolutely foolproof for all real symmetric matrices. In particular, it returns the small eigenvalues with better relative accuracy than methods that first reduce the matrix to tridiagonal form. For matrices of order greater than about 10, however, the algorithm is slower, by a significant constant factor, than the *QR* method we shall give in §11.4. However, the Jacobi algorithm is much simpler than the more efficient methods. We thus recommend it for matrices of moderate order, where expense is not a major consideration.

The basic Jacobi rotation $\mathbf{P}_{pq}$ is a matrix of the form

$$\mathbf{P}_{pq} = \begin{bmatrix} 1 & & & & \\ & \cdots & & & \\ & & c & \cdots & s \\ & & \vdots & 1 & \vdots \\ & & -s & \cdots & c \\ & & & & \cdots \\ & & & & & 1 \end{bmatrix} \quad (11.1.1)$$

Here all the diagonal elements are unity except for the two elements c in rows (and columns) p and q . All off-diagonal elements are zero except the two elements s and $-s$. The numbers c and s are the cosine and sine of a rotation angle ϕ , so $c^2 + s^2 = 1$.

A plane rotation such as (11.1.1) is used to transform the matrix $\mathbf{A}$ according to

$$\mathbf{A}' = \mathbf{P}_{pq}^T \cdot \mathbf{A} \cdot \mathbf{P}_{pq} \quad (11.1.2)$$

Now, $\mathbf{P}_{pq}^T \cdot \mathbf{A}$ changes only rows p and q of $\mathbf{A}$, while $\mathbf{A} \cdot \mathbf{P}_{pq}$ changes only columns p and q . Notice that the subscripts p and q do not denote components of $\mathbf{P}_{pq}$, but rather label which kind of rotation the matrix is, i.e., which rows and columns it affects. Thus the changed elements of $\mathbf{A}$ in (11.1.2) are only in rows p and q , and columns p and q , as indicated below:

$$\mathbf{A}' = \begin{bmatrix} & & a'_{0p} & & a'_{0q} & & \\ & & \vdots & & \vdots & & \\ a'_{p0} & \cdots & a'_{pp} & \cdots & a'_{pq} & \cdots & a'_{p,n-1} \\ & & \vdots & & \vdots & & \\ a'_{q0} & \cdots & a'_{qp} & \cdots & a'_{qq} & \cdots & a'_{q,n-1} \\ & & \vdots & & \vdots & & \\ & & a'_{n-1,p} & & a'_{n-1,q} & & \end{bmatrix} \quad (11.1.3)$$

Multiplying out equation (11.1.2) and using the symmetry of $\mathbf{A}$, we get the explicit formulas

$$\left. \begin{aligned} a'_{rp} &= ca_{rp} - sa_{rq} \\ a'_{rq} &= ca_{rq} + sa_{rp} \end{aligned} \right\} \quad r \neq p, r \neq q \quad (11.1.4)$$

$$a'_{pp} = c^2 a_{pp} + s^2 a_{qq} - 2sca_{pq} \quad (11.1.5)$$

$$a'_{qq} = s^2 a_{pp} + c^2 a_{qq} + 2sca_{pq} \quad (11.1.6)$$

$$a'_{pq} = (c^2 - s^2)a_{pq} + sc(a_{pp} - a_{qq}) \quad (11.1.7)$$